

OPTION B INSTITUTE FAMILY

M10 - DOCKER + AIRFLOW

Standalone Zero-to-Job-Ready Data Engineer 2026

BEGINNER-FIRST MASTER REPAIR RC4

19 canonical lessons | full 16-stage teacher contract | protected attempt-first review

This learner book is rebuilt from the frozen canonical technical material. It is not a front-matter patch: every lesson is re-laid out into the shared institute teaching grammar while preserving the course competency, source route, guided work, independent proof and validation boundary.

This book is your teacher

Use the book in order. External videos and official docs are visual companions, not hidden prerequisites. Watch or read only the assigned scope, then close it before the independent task.

The 16-stage lesson loop

Why it matters -> What you learn -> Prerequisite -> Picture it -> New words -> Useful source -> Follow once -> Expected result -> Why it worked -> Beginner mistakes -> Recovery -> Independent task -> Validate -> Teach it back -> Protected review + changed retry -> Evidence/mastery + exact next route.

Attempt-first protection

Never open a protected lesson review before saving a genuine independent attempt. If repair is needed, compare only the weak idea, close the review, solve a changed retry, then validate again. A solution seen during review is repair evidence, not mastery.

What counts as mastery

A reproducible artifact or explanation, an independent validation control, preserved failure/recovery evidence, a changed transfer task completed without the worked answer open, and an own-words explanation. Percent-complete is not a substitute.

How to use your devices

Phone: assigned video/reference, reading, revision. Laptop/PC: every real command, query, file, workbook, code task, validation and evidence artifact. Install only what the current lesson asks for.

Contents / Index

This book is your teacher	2
Contents / Index	3
Prerequisite / tool map	4
Practice / review / Gate route	4
DE10-01 - Why containers exist	5
DE10-02 - Images and containers	7
DE10-03 - Writing a Dockerfile	9
DE10-04 - Ports	11
DE10-05 - Volumes and persistence	13
DE10-06 - Networks and service communication	15
DE10-07 - Compose, configuration and secrets	17
DE10-08 - Why orchestration exists	19
DE10-09 - Airflow architecture and UI	21
DE10-10 - DAGs, tasks and dependencies	23
DE10-11 - TaskFlow, operators and task communication	25
DE10-12 - Scheduling and data intervals	27
DE10-13 - Retries, timeouts and failure behavior	29
DE10-14 - Parameters and task communication	31
DE10-15 - Connections, variables and secrets	33
DE10-16 - Sensors and waiting	35
DE10-17 - Logs and debugging	37
DE10-18 - Catchup, backfills and reprocessing	39
DE10-19 - Integration and production-style orchestration	41
Module mastery checklist + revision quick reference	43

Prerequisite / tool map

Package work with Docker and orchestrate reliable workflows with Airflow concepts and evidence.

Item	Rule
Entry prerequisite	Previous module/Gate evidence must be stable.
Primary device	PC/laptop for hands-on work; phone is support only.
External sources	Optional/supporting unless the lesson explicitly marks an official current reference.
Evidence	Module practical + fresh retest + saved evidence; no critical unresolved failure.

Practice / review / Gate route

Lesson teaching -> follow once -> independent attempt -> validate -> save evidence. Only after the genuine attempt may protected review open. If weak: targeted repair -> close review -> changed retry -> revalidate. Finish the module practical/retest route before any downstream Gate that depends on this module.

DE10-01 - Why containers exist

1. Why it matters

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

2. What you are learning

Explain what a container isolates and what it does not.

3. Prerequisite / starting state

Starting state: no prior technical knowledge is assumed beyond the module Start Here instructions.

4. Picture it in your head

An image is packaged filesystem/config metadata; a container is a running instance that still uses the host kernel. Containers improve reproducibility, not magical security or persistence.

5. New words before we use them

An image is packaged filesystem/config metadata; a container is a running instance that still uses the host kernel. Containers improve reproducibility, not magical security or persistence.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Docker & PostgreSQL DE workshop

Assigned scope: Use the matching Docker / Dockerfile / networking / Compose chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Compare running worker.py on the host with the declared Docker environment. Compare running worker.py on the host with the declared Docker environment. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If you call a container a VM, restate kernel sharing and lifecycle.

11. If you get stuck

If you call a container a VM, restate kernel sharing and lifecycle.

12. Small independent task

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

13. Validate

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

16. Evidence / mastery + exact next route

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off. Exact next route: DE10-02

DE10-02 - Images and containers

1. Why it matters

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

2. What you are learning

Distinguish immutable image layers from mutable container runtime state.

3. Prerequisite / starting state

Prerequisite: DE10-01 mastered with saved evidence.

4. Picture it in your head

Build once, run many. Container changes disappear when the container is replaced unless persisted externally.

5. New words before we use them

Build once, run many. Container changes disappear when the container is replaced unless persisted externally.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Docker & PostgreSQL DE workshop

Assigned scope: Use the matching Docker / Dockerfile / networking / Compose chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Trace FROM/COPY/RUN into image, then CMD into a container process. Trace FROM/COPY/RUN into image, then CMD into a container process. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If you rely on changes made interactively inside a container, rebuild the image instead.

11. If you get stuck

If you rely on changes made interactively inside a container, rebuild the image instead.

12. Small independent task

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

13. Validate

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

16. Evidence / mastery + exact next route

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off. Exact next route: DE10-03

DE10-03 - Writing a Dockerfile

1. Why it matters

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

2. What you are learning

Write a small, deterministic Dockerfile with a clear base, workdir, copy/install order and non-root runtime.

3. Prerequisite / starting state

Prerequisite: DE10-02 mastered with saved evidence.

4. Picture it in your head

Cache-friendly build order copies dependency metadata before changing app code. Pin deliberately when reproducibility matters.

5. New words before we use them

Cache-friendly build order copies dependency metadata before changing app code. Pin deliberately when reproducibility matters. Avoid secrets in build context/layers.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Docker & PostgreSQL DE workshop

Assigned scope: Use the matching Docker / Dockerfile / networking / Compose chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Inspect the provided Python 3.13-slim Dockerfile and .dockerignore. Inspect the provided Python 3.13-slim Dockerfile and .dockerignore. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If builds leak local files, inspect build context and .dockerignore.

11. If you get stuck

If builds leak local files, inspect build context and .dockerignore.

12. Small independent task

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

13. Validate

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

16. Evidence / mastery + exact next route

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off. Exact next route: DE10-04

DE10-04 - Ports

1. Why it matters

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

2. What you are learning

Explain container ports, EXPOSE, host publishing and loopback binding.

3. Prerequisite / starting state

Prerequisite: DE10-03 mastered with saved evidence.

4. Picture it in your head

EXPOSE documents intended container ports; it does not publish them. Host publication maps host:container and binding to 127.0.0.1 limits local exposure.

5. New words before we use them

EXPOSE documents intended container ports; it does not publish them. Host publication maps host:container and binding to 127.0.0.1 limits local exposure.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Docker & PostgreSQL DE workshop

Assigned scope: Use the matching Docker / Dockerfile / networking / Compose chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Read 127.0.0.1:8080:8080 in compose.yaml. Read 127.0.0.1:8080:8080 in compose.yaml. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If localhost cannot connect, separate app listen address, container port, published host port and firewall.

11. If you get stuck

If localhost cannot connect, separate app listen address, container port, published host port and firewall.

12. Small independent task

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

13. Validate

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

16. Evidence / mastery + exact next route

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off. Exact next route: DE10-05

DE10-05 - Volumes and persistence

1. Why it matters

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

2. What you are learning

Choose a named volume when data must outlive container replacement.

3. Prerequisite / starting state

Prerequisite: DE10-04 mastered with saved evidence.

4. Picture it in your head

Container writable layers are ephemeral. Volumes are engine-managed persistent stores; bind mounts expose host paths and fit different use cases.

5. New words before we use them

Container writable layers are ephemeral. Volumes are engine-managed persistent stores; bind mounts expose host paths and fit different use cases.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Docker & PostgreSQL DE workshop

Assigned scope: Use the matching Docker / Dockerfile / networking / Compose chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Trace pgdata to /var/lib/postgresql/data. Trace pgdata to /var/lib/postgresql/data. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If data vanishes after recreation, prove where it was written.

11. If you get stuck

If data vanishes after recreation, prove where it was written.

12. Small independent task

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

13. Validate

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

16. Evidence / mastery + exact next route

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off. Exact next route: DE10-06

DE10-06 - Networks and service communication

1. Why it matters

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

2. What you are learning

Use service-name DNS for container-to-container communication.

3. Prerequisite / starting state

Prerequisite: DE10-05 mastered with saved evidence.

4. Picture it in your head

Compose creates a project network by default; services on it resolve each other by service name. Do not hardcode changing container IPs.

5. New words before we use them

Compose creates a project network by default; services on it resolve each other by service name. Do not hardcode changing container IPs.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Docker & PostgreSQL DE workshop

Assigned scope: Use the matching Docker / Dockerfile / networking / Compose chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

The api uses DB_HOST=db and both join backend. The api uses DB_HOST=db and both join backend. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If service A cannot reach B, inspect network membership, service name and listening port before host port mappings.

11. If you get stuck

If service A cannot reach B, inspect network membership, service name and listening port before host port mappings.

12. Small independent task

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

13. Validate

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

16. Evidence / mastery + exact next route

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off. Exact next route: DE10-07

DE10-07 - Compose, configuration and secrets

1. Why it matters

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

2. What you are learning

Define a multi-service stack while separating normal configuration from secrets.

3. Prerequisite / starting state

Prerequisite: DE10-06 mastered with saved evidence.

4. Picture it in your head

Compose manages services, networks and volumes. Sensitive values should not be committed in source; use secrets or an appropriate secret backend.

5. New words before we use them

Compose manages services, networks and volumes. Sensitive values should not be committed in source; use secrets or an appropriate secret backend.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Docker & PostgreSQL DE workshop

Assigned scope: Use the matching Docker / Dockerfile / networking / Compose chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Inspect POSTGRES_PASSWORD_FILE and per-service secret grant. Inspect POSTGRES_PASSWORD_FILE and per-service secret grant. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If a secret appears in Dockerfile, Git or logs, rotate it and redesign injection.

11. If you get stuck

If a secret appears in Dockerfile, Git or logs, rotate it and redesign injection.

12. Small independent task

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

13. Validate

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

16. Evidence / mastery + exact next route

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off. Exact next route: DE10-08

DE10-08 - Why orchestration exists

1. Why it matters

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

2. What you are learning

Explain why orchestration exists above scripts/cron.

3. Prerequisite / starting state

Prerequisite: DE10-07 mastered with saved evidence.

4. Picture it in your head

Orchestration models dependencies, schedules, state, retries, observability and reruns. It coordinates work; it does not make unsafe tasks idempotent.

5. New words before we use them

Orchestration models dependencies, schedules, state, retries, observability and reruns. It coordinates work; it does not make unsafe tasks idempotent.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: For backfills/reprocessing use current Astronomer docs; for DAG testing use current testing docs.

[Open source](#)

7. Follow with me once

Map M09 pipeline steps into wait -> extract -> validate -> transform -> load -> reconcile. Map M09 pipeline steps into wait -> extract -> validate -> transform -> load -> reconcile. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If retrying the DAG duplicates data, repair task/data semantics before increasing retries.

11. If you get stuck

If retrying the DAG duplicates data, repair task/data semantics before increasing retries.

12. Small independent task

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

13. Validate

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

16. Evidence / mastery + exact next route

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off. Exact next route: DE10-09

DE10-09 - Airflow architecture and UI

1. Why it matters

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

2. What you are learning

Identify scheduler, DAG processing, executor/workers, metadata/API/UI roles at a practical level.

3. Prerequisite / starting state

Prerequisite: DE10-08 mastered with saved evidence.

4. Picture it in your head

The scheduler decides runnable work; executors/workers run tasks; metadata tracks state; UI/API expose control/inspection. Airflow 3 uses a stable Task SDK/public interface.

5. New words before we use them

The scheduler decides runnable work; executors/workers run tasks; metadata tracks state; UI/API expose control/inspection. Airflow 3 uses a stable Task SDK/public interface.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: For backfills/reprocessing use current Astronomer docs; for DAG testing use current testing docs.

[Open source](#)

7. Follow with me once

Use the current UI reference to locate DAG runs, task logs and task states. Use the current UI reference to locate DAG runs, task logs and task states. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If UI says failed, read the specific task/run/log rather than restarting everything.

11. If you get stuck

If UI says failed, read the specific task/run/log rather than restarting everything.

12. Small independent task

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

13. Validate

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

16. Evidence / mastery + exact next route

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off. Exact next route: DE10-10

DE10-10 - DAGs, tasks and dependencies

1. Why it matters

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

2. What you are learning

Define a DAG as tasks plus dependency rules, not as a giant script.

3. Prerequisite / starting state

Prerequisite: DE10-09 mastered with saved evidence.

4. Picture it in your head

Tasks should have clear inputs/outputs, bounded side effects and explicit dependencies. Independent tasks can run concurrently when safe.

5. New words before we use them

Tasks should have clear inputs/outputs, bounded side effects and explicit dependencies. Independent tasks can run concurrently when safe.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: For backfills/reprocessing use current Astronomer docs; for DAG testing use current testing docs.

[Open source](#)

7. Follow with me once

Read m10_daily_orders.py dependency chain. Read m10_daily_orders.py dependency chain.

Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If ordering is only hidden in code, make dependency structure explicit.

11. If you get stuck

If ordering is only hidden in code, make dependency structure explicit.

12. Small independent task

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

13. Validate

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

16. Evidence / mastery + exact next route

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off. Exact next route: DE10-11

DE10-11 - TaskFlow, operators and task communication

1. Why it matters

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

2. What you are learning

Use TaskFlow for Python task composition and keep task communication small.

3. Prerequisite / starting state

Prerequisite: DE10-10 mastered with saved evidence.

4. Picture it in your head

TaskFlow automatically wires returned values through Airflow task communication. XCom-style payloads are control/small metadata, not a replacement for data storage.

5. New words before we use them

TaskFlow automatically wires returned values through Airflow task communication. XCom-style payloads are control/small metadata, not a replacement for data storage.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: For backfills/reprocessing use current Astronomer docs; for DAG testing use current testing docs.

[Open source](#)

7. Follow with me once

Extract returns rows, load returns a summary, reconcile consumes summary. Extract returns rows, load returns a summary, reconcile consumes summary. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If huge data is passed between tasks, persist it externally and pass a reference.

11. If you get stuck

If huge data is passed between tasks, persist it externally and pass a reference.

12. Small independent task

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

13. Validate

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

16. Evidence / mastery + exact next route

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off. Exact next route: DE10-12

DE10-12 - Scheduling and data intervals

1. Why it matters

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

2. What you are learning

Reason about schedule, logical date/data interval, start date and manual runs.

3. Prerequisite / starting state

Prerequisite: DE10-11 mastered with saved evidence.

4. Picture it in your head

A scheduled run represents a data interval, not simply the wall-clock instant the task starts. Use UTC-aware reasoning and explicit interval-based data queries.

5. New words before we use them

A scheduled run represents a data interval, not simply the wall-clock instant the task starts. Use UTC-aware reasoning and explicit interval-based data queries.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: For backfills/reprocessing use current Astronomer docs; for DAG testing use current testing docs.

[Open source](#)

7. Follow with me once

For a daily DAG, state which interval a run should process. For a daily DAG, state which interval a run should process. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If a run reads now() blindly, backfills may process the wrong slice.

11. If you get stuck

If a run reads now() blindly, backfills may process the wrong slice.

12. Small independent task

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

13. Validate

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

16. Evidence / mastery + exact next route

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off. Exact next route: DE10-13

DE10-13 - Retries, timeouts and failure behavior

1. Why it matters

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

2. What you are learning

Configure retries for transient failures and timeouts for bounded execution.

3. Prerequisite / starting state

Prerequisite: DE10-12 mastered with saved evidence.

4. Picture it in your head

Retry only plausibly transient work. A retry is safe only when side effects are idempotent/transactional.

5. New words before we use them

Retry only plausibly transient work. A retry is safe only when side effects are idempotent/transactional. `execution_timeout` bounds a task attempt.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: For backfills/reprocessing use current Astronomer docs; for DAG testing use current testing docs.

[Open source](#)

7. Follow with me once

Simulator extract fails twice then succeeds on attempt 3; validation failure does not retry blindly.

Simulator extract fails twice then succeeds on attempt 3; validation failure does not retry blindly.

Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If retries amplify damage, stop and repair idempotency or classify permanent errors.

11. If you get stuck

If retries amplify damage, stop and repair idempotency or classify permanent errors.

12. Small independent task

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

13. Validate

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

16. Evidence / mastery + exact next route

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off. Exact next route: DE10-14

DE10-14 - Parameters and task communication

1. Why it matters

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

2. What you are learning

Use runtime Params for validated run configuration and task communication for outputs.

3. Prerequisite / starting state

Prerequisite: DE10-13 mastered with saved evidence.

4. Picture it in your head

Params are configuration supplied/defaulted for a run and JSON-Schema validated. Task outputs carry produced metadata/results.

5. New words before we use them

Params are configuration supplied/defaulted for a run and JSON-Schema validated. Task outputs carry produced metadata/results. Do not confuse them.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: For backfills/reprocessing use current Astronomer docs; for DAG testing use current testing docs.

[Open source](#)

7. Follow with me once

min_amount is a validated DAG Param; task results form the simulator xcom dictionary. min_amount is a validated DAG Param; task results form the simulator xcom dictionary. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If a parameter changes business scope, record it in run metadata.

11. If you get stuck

If a parameter changes business scope, record it in run metadata.

12. Small independent task

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

13. Validate

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

16. Evidence / mastery + exact next route

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off. Exact next route: DE10-15

DE10-15 - Connections, variables and secrets

1. Why it matters

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

2. What you are learning

Keep connection credentials and sensitive Variables out of DAG source.

3. Prerequisite / starting state

Prerequisite: DE10-14 mastered with saved evidence.

4. Picture it in your head

Connections identify external systems; secrets can come from environment/metastore/external backends. Masking helps but is not permission control.

5. New words before we use them

Connections identify external systems; secrets can come from environment/metastore/external backends. Masking helps but is not permission control.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: For backfills/reprocessing use current Astronomer docs; for DAG testing use current testing docs.

[Open source](#)

7. Follow with me once

Explain why conn_id belongs in DAG code but password does not. Explain why conn_id belongs in DAG code but password does not. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If plaintext credentials are committed, treat it as exposure and rotate.

11. If you get stuck

If plaintext credentials are committed, treat it as exposure and rotate.

12. Small independent task

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

13. Validate

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

16. Evidence / mastery + exact next route

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off. Exact next route: DE10-16

DE10-16 - Sensors and waiting

1. Why it matters

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

2. What you are learning

Wait for external conditions without wasting worker capacity indefinitely.

3. Prerequisite / starting state

Prerequisite: DE10-15 mastered with saved evidence.

4. Picture it in your head

Sensors poll/wait. Reschedule or deferrable patterns can release worker resources; every wait needs a timeout/failure policy.

5. New words before we use them

Sensors poll/wait. Reschedule or deferrable patterns can release worker resources; every wait needs a timeout/failure policy.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: For backfills/reprocessing use current Astronomer docs; for DAG testing use current testing docs.

[Open source](#)

7. Follow with me once

Simulator wait_input has bounded polls and fails downstream safely. Simulator wait_input has bounded polls and fails downstream safely. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If a sensor waits forever, add timeout and decide skip/fail/escalate behavior.

11. If you get stuck

If a sensor waits forever, add timeout and decide skip/fail/escalate behavior.

12. Small independent task

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

13. Validate

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

16. Evidence / mastery + exact next route

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off. Exact next route: DE10-17

DE10-17 - Logs and debugging

1. Why it matters

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

2. What you are learning

Debug from task state and logs using a repeatable ladder.

3. Prerequisite / starting state

Prerequisite: DE10-16 mastered with saved evidence.

4. Picture it in your head

Start with failing task -> attempt -> exception -> input/config -> upstream state -> environment. Logs should include run/task identifiers but never secrets.

5. New words before we use them

Start with failing task -> attempt -> exception -> input/config -> upstream state -> environment. Logs should include run/task identifiers but never secrets.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: For backfills/reprocessing use current Astronomer docs; for DAG testing use current testing docs.

[Open source](#)

7. Follow with me once

Read simulator logs for transient retries and validation failure. Read simulator logs for transient retries and validation failure. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If you rerun without identifying cause, you may hide an intermittent defect.

11. If you get stuck

If you rerun without identifying cause, you may hide an intermittent defect.

12. Small independent task

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

13. Validate

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

16. Evidence / mastery + exact next route

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off. Exact next route: DE10-18

DE10-18 - Catchup, backfills and reprocessing

1. Why it matters

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

2. What you are learning

Backfill historical intervals without changing business meaning or duplicating outputs.

3. Prerequisite / starting state

Prerequisite: DE10-17 mastered with saved evidence.

4. Picture it in your head

Backfills recreate runs for past data intervals. Tasks must use interval inputs and idempotent writes.

5. New words before we use them

Backfills recreate runs for past data intervals. Tasks must use interval inputs and idempotent writes. Catchup is a scheduling choice; backfill is an explicit operation.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: For backfills/reprocessing use current Astronomer docs; for DAG testing use current testing docs.

[Open source](#)

7. Follow with me once

Simulator backfills three named dates and produces independent summaries. Simulator backfills three named dates and produces independent summaries. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If historical rerun uses current date/config accidentally, stop and make interval/config explicit.

11. If you get stuck

If historical rerun uses current date/config accidentally, stop and make interval/config explicit.

12. Small independent task

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

13. Validate

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

16. Evidence / mastery + exact next route

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off. Exact next route: DE10-19

DE10-19 - Integration and production-style orchestration

1. Why it matters

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

2. What you are learning

Defend an end-to-end orchestration design including deployment, state, recovery and observability.

3. Prerequisite / starting state

Prerequisite: DE10-18 mastered with saved evidence.

4. Picture it in your head

Production readiness combines reproducible runtime, clear dependencies, safe secrets, bounded retries/waits, idempotent data operations, reconciliation, logs and tested reprocessing.

5. New words before we use them

Production readiness combines reproducible runtime, clear dependencies, safe secrets, bounded retries/waits, idempotent data operations, reconciliation, logs and tested reprocessing.

6. Useful video / source

Source: Astronomer — Airflow 101: Essential Tips for Beginners

Assigned scope: Use as visual concept/demo; Teaching Book + current docs cover the exact Airflow version

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: For backfills/reprocessing use current Astronomer docs; for DAG testing use current testing docs.

[Open source](#)

7. Follow with me once

Combine Docker static lab with Airflow DAG + simulator evidence; identify what still requires real runtime. Combine Docker static lab with Airflow DAG + simulator evidence; identify what still requires real runtime. Reproduce the matching course asset once and save concrete evidence.

8. Expected result / what you should see

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If a diagram cannot explain failure/recovery and evidence, it is not an operational design.

11. If you get stuck

If a diagram cannot explain failure/recovery and evidence, it is not an operational design.

12. Small independent task

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

13. Validate

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence. Save a count, schema/key check, reconciliation, observed state, test, or other independent control appropriate to the lesson.

14. Teach it back

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery. Changed retry from the canonical lesson: Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

16. Evidence / mastery + exact next route

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off. Exact next route: Proceed to the module practical/Gate route after the module mastery checklist.

Module mastery checklist + revision quick reference

- ☐ Every lesson has a saved genuine independent attempt.
- ☐ Every lesson has at least one independent validation/control appropriate to the tool.
- ☐ Protected reviews were opened only after attempts; any weak lesson has a changed retry after review was closed.
- ☐ You can teach back each lesson in your own words and name one realistic failure mode.
- ☐ Required module practical/retest evidence is saved and no critical defect is unresolved.
- ☐ D+1, D+4, D+10 and D+30 revision events are scheduled for weak or high-value competencies.

Revision cadence

D+1: fresh micro-task. D+4: mixed retrieval. D+10: changed/failure case. D+30: interview/project retrieval. Always attempt from memory before reopening a source.

Release boundary

This book is a static learner artifact. It does not prove live tool/cloud execution or a protected Gate pass. Runtime evidence must be recorded separately when the curriculum requires it.